

Llama1模型修改

Transformer Decoder 架构，做了以下修改：

1. 和GPT3一样将Normalization从每个子层的输出位置移动到了输入位置。 *求稳*
2. 将Layer Norm 改为 RMS Norm。 *求快*
3. 采用旋转位置编码。 *求懂*
4. 采用silu激活函数。 *求聪明*

1. 将 Normalization (归一化) 从输出位置移动到了输入位置 (Pre-Norm)

- 这是什么意思：
在神经网络中，“归一化”就像是给数据做“体检和安检”，把大小不一、乱七八糟的数据调整到一个标准的范围内，防止数据在传递过程中“爆炸”或“消失”。
 - 老办法 (Post-Norm)：原来的 Transformer 是等数据经过一系列复杂的计算（子层）之后，输出前才做归一化。就像是工厂里，零件加工完了才去质检。
 - Llama 的做法 (Pre-Norm)：在数据进入复杂计算之前，先做归一化。就像是原料刚进车间，先做标准化处理，再加工。
- 为什么要这么做：
为了让模型更稳定，不容易“炸掉”。当模型层数非常深（几十上百层）时，老办法很容易导致信号传不到深层，或者信号过导致计算崩溃（梯度消失或爆炸）。放在输入端提前规范化，能保证每一层接收到的数据都是“健康”的，这让大模型的训练变得极其稳定。

2. 将 Layer Norm 改为 RMS Norm

- 这是什么意思：
这是一种“偷懒但高效”的归一化算法升级。
$$RMS\ Norm = \frac{\sqrt{\text{mean}(x^2) + \epsilon}}{\sqrt{d}} * y$$
 - 老办法 (Layer Norm)：算得很仔细。它要把数据“减去平均值”（居中），然后再“除以标准差”（缩放）。这好比比老师调整全班成绩，既要算平均分把大家拉回及格线，又要算方差缩小贫富差距。
 - Llama 的做法 (RMS Norm)：简化了计算。科学家发现，其实“减去平均值”这一步对大模型来说没什么用。所以 RMS Norm 只做缩放，不管平均值。
- 为什么要这么做：
为了省时间和算力，提速！砍掉了一半的计算步骤，模型跑起来更快了，而且事实证明，这样“偷工减料”后，模型的聪明程度完全没有下降。

3. 采用旋转位置编码 (RoPE - Rotary Position Embedding)

- 这是什么意思：
模型本身是个“瞎子”，它同时看到所有词，不知道谁在前面谁在后面，所以我们需要给词贴上位置标签（位置编码）。
 - 老办法 (绝对位置)：给每个词贴死标签。比如“我(1) 爱(2) 吃(3) 苹果(4)”。
 - Llama 的做法 (旋转位置/RoPE)：它不仅关心词的绝对位置，更关心词汇之间的 相对距离。它通过一种数学上的“旋转”操作，让词语的表达自带方向感。
 - 比喻：就像时钟的指针。我不告诉你现在是几点（绝对时间），但我告诉你分针和时针之间差了30度（相对位置）。不管在哪个小时，差30度的关系是不变的。
- 为什么要这么做：
为了 更好地理解语言的上下文。语言中词汇的关系往往取决于它们离得有多近（比如“不”和“好”连在一起就是否定）。RoPE 能极好地捕捉这种相对距离，而且它能让模型在遇到比训练时更长的文章时，依然保持很好的理解能力（泛化能力强）。

RoPE

$$q = \begin{bmatrix} q_1, & q_2 \end{bmatrix} \quad \text{attention}(q, k) = q \cdot k = qk^T \quad \text{没有位置信息}$$
$$k = \begin{bmatrix} k_1, & k_2 \end{bmatrix}$$

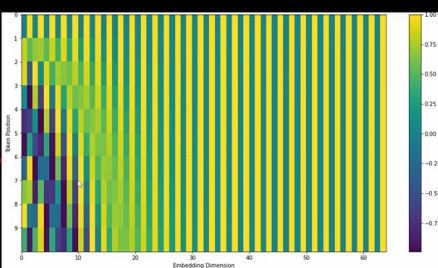
旋转后再点乘

$$\begin{aligned} qR(m) \cdot kR(n) &= qR(m)R(n)^T k^T \\ &= qR(m)R(-n)k^T \\ &= qR(m - n)k^T \quad \text{带有相对位置信息} \end{aligned}$$

R(mf) =

$$\begin{bmatrix} \cos mf_0 & \sin mf_0 & 0 & 0 & \dots & 0 & 0 \\ -\sin mf_0 & \cos mf_0 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \cos mf_1 & \sin mf_1 & \dots & 0 & 0 \\ 0 & 0 & -\sin mf_1 & \cos mf_1 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & \cos mf_{\frac{d}{2}-1} & \sin mf_{\frac{d}{2}-1} \\ 0 & 0 & 0 & 0 & \dots & -\sin mf_{\frac{d}{2}-1} & \cos mf_{\frac{d}{2}-1} \end{bmatrix}$$

$$f_i = \frac{1}{10000^{\frac{2i}{d}}}$$



* 旋转后 类内升 类间降

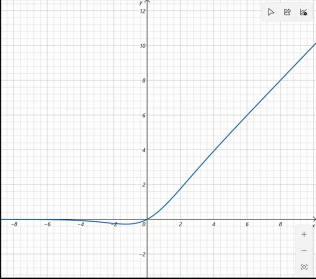
类升

$$f_i = \frac{1}{10000^{\frac{2i}{d}}}$$

4. 采用 SiLU 激活函数 (SwiGLU 的一部分)

- 这是什么意思：
激活函数是神经网络的“灵魂”，它决定了神经元在接收到信号后，是否要“激活”并把信号传给下一层。
 - 老办法 (ReLU)：非常简单粗暴。如果信号是负数，直接切断（变成0）；如果是正数，原样输出。就像一个硬邦邦的开关，要么关死，要么打开。
 - Llama的做法 (SiLU)：它是一条平滑的曲线。对于正数，它像 ReLU 一样放行；但对于负数，它不会一刀切变成0，而是保留了一点点负值，并且拐角处非常圆滑。就像一个平滑的旋钮开关。
- 为什么要这么做：
为了让模型更聪明，学得更好。这种平滑的曲线在数字计算（求导）时非常丝滑，信息传递更顺畅。在实际测试中，换成这种激活函数后，模型的各项考试成绩（性能）都有了明显的提升。

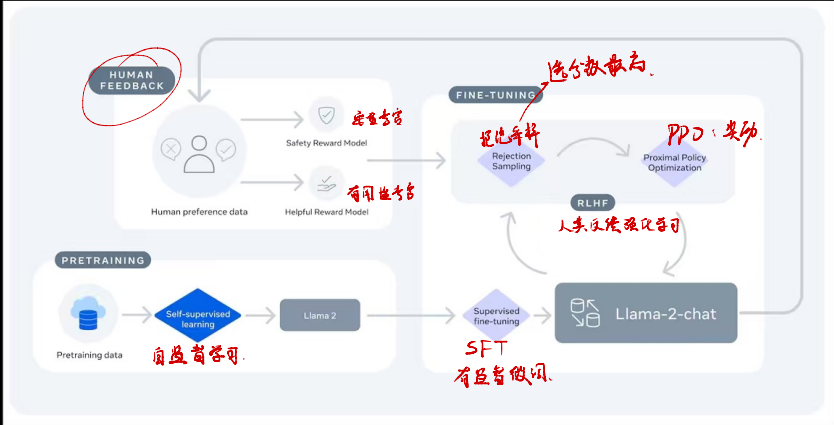
silu



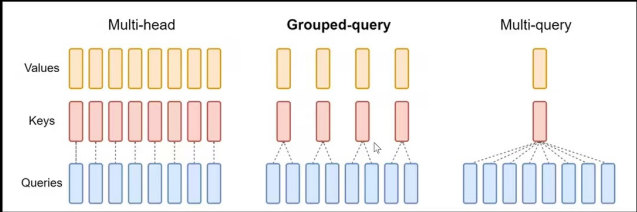
$$\text{silu} = x * \text{sigmoid}(x) = \frac{x}{1 + e^{-x}}$$

可以提升精度，
计算指数函数，计算代价较高

Llama2



GQA(Group Query Attention)



这张图说明了大模型在内存优化上的演进：
MHA（多头）太占显存 -> MQA（单头共享）变笨了 -> GQA（分组共享）既聪明又省显存。

1. 最大的质变：从“半成品基座”到“成品聊天助手” (对应你问的第一张图)

- **Llama 1**：当时 Meta 只发布了第一阶段的“预训练基础模型”（只会词语接龙的书呆子）。它没有经过微调，不能直接用来对话。后来是斯坦福等学术界的社区大神们，自己弄了点数据把它微调成了能对话的“羊驼 (Alpaca)”。
- **Llama 2**：Meta 这次官方亲自下场，花了海量的金钱和人力，走完了 **SFT**（监督微调）和 **RLHF**（人类反馈强化学习）全流程，直接发布了 **Llama-2-Chat** 版本。它的情商、安全性和对话能力有了质的飞跃。

2. 独家秘方：Ghost Attention (幽灵注意力 - GAtt)

- 以前的大模型有个通病：比如你第一句跟它说“接下来的对话请你一直扮演李白”，聊到第五句它就忘了，变成正常机器人了。
- **Llama 2 引入了 GAtt 技术**：这是在微调阶段加的一个巧妙设计，相当于把你的初始指令像“幽灵”一样悄悄绑定在后面的每一轮对话里。让 Llama 2 在极长的多轮对话中，依然能死死记住你最初定下的规矩。

3. 阅读量大增：预训练数据多了 40%

- **Llama 1**：训练数据大概是 1.4 万亿个 Token（词元）。
- **Llama 2**：猛增到了 **2 万亿 (2T) 个 Token**。而且 Meta 对数据做了更严格的清洗，删除了包含大量个人隐私的网站，这让模型不仅见多识广，而且“底子”更干净。

4. 记忆力翻倍：上下文窗口 (Context Window) 从 2048 变 4096

- **Llama 1**：只能记住 2048 个 Token（大约一两千字），一篇文章长一点它就读不完了。
- **Llama 2**：直接翻倍提升到 **4096 个 Token**。它可以一次性阅读并理解更长的文档和代码了。（注：现在的最新模型甚至能达到 128k 甚至百万级别，但在当时 4096 是个大升级）。

Llma3

模型架构：

字典从3万2000个Token扩充4倍，达到了12万8千。提高推理效率。
原来一个中文被编码为多个token，现在只需要一个token。

所有模型，包括8B模型也用GQA

序列长度从4096到了8192。

训练数据：

15T的训练Token，全部来自公开数据。是Llama2的7倍大小。代码数据多了4倍。

5%高质量非英语数据，涵盖30多种语言。

对数据进行了清洗过滤，Llama2生成训练数据来帮助训练文本质量分类器。

微调阶段除了开源数据集，还人工标注了1000万样本。

	Training Data	Params	Context length	GQA	Token count	Knowledge cutoff
Llama 3	A new mix of publicly available online data.	8B	8k	Yes	15T+	March, 2023
		70B	8k	Yes		December, 2023

2. Llama 4 MoE 架构的核心技术细节

Llama 4 的 MoE 并不只是简单地把多个网络拼在一起，Meta 在底层架构上做了非常深度的修改：

- 极端的“稀疏激活”（Sparse Activation）：以 Maverick 为例，虽然它有 4000 亿的总参数，但单次仅激活 170 亿参数（占比仅约 4.25%）。这种设计打破了传统的“缩放定律”（Scaling Laws），让模型在拥有庞大知识储备的同时，推理速度和内存计算压力大幅下降。
- 交错式设计（Interleaved MoE/Dense Layers）：Llama 4 并没有让所有层都是 MoE 层。它采用了交错设计，将传统的稠密层（Dense Layers）与 MoE 层交织在一起，既保证了基础语义的稳定传递，又利用 MoE 层实现了专业知识的路由。
- “共享 + 路由”专家机制（Shared + Routed Experts）：特别是在 Maverick 这种多达 128 个专家的模型中，Llama 4 引入了创新的专家设计：
 - 共享专家（Shared Expert）：负责处理所有通用知识和基础逻辑，每个 Token 都会经过它。
 - 路由专家（Routed Experts）：路由器（Router）会根据当前词汇的特性，从剩下的专家中挑选最合适的 1~2 个专家进行专门处理（比如遇到代码就找代码专家，遇到数学公式就找数学专家）。
- 高性能的 SwiGLU 专家节点：其内部的每一个“专家”本质上都是一个采用 SwiGLU 激活函数的前馈神经网络，这继承并优化了前代 Llama 优秀的底层运算效率。

3. MoE 带来的两大附带“神技”

除了 MoE 本身的路由机制，Llama 4 的架构还配合了另外两项颠覆性创新，使其威力倍增：

- 原生多模态（Early Fusion，早期融合）：过去很多模型是“先看图，翻译成文字，再给大模型处理”。而 Llama 4 采用了 Early Fusion，视觉 Token 和文本 Token 从一开始就被送入同一个 MoE 骨干网络中。无论是看图、读文字还是看视频，底层的 128 个专家都能统一处理这些跨模态信息。
- iRoPE（交锚旋转位置编码）与千万级上下文：结合 MoE 的高效算力，Llama 4 引入了全新的 iRoPE 架构（交锚注意力层+无位置编码+推理期温度缩放），这让它的上下文窗口直接飙升到了 1000 万 Token（10M Context），可以直接吞下整个大型 GitHub 代码库进行分析，这在 Llama 3 时代是完全无法想象的。

总结一下 Llama 4 的 MoE：

在 2026 年的今天回望，如果说 Llama 3 是 Transformer 稠密模型的大成之作，那么 Llama 4 的 MoE 架构就是 Meta 向更深层次“类脑结构”演进的开始。通过 128 个细分专家（Maverick）和原生的多模态融合，它彻底改变了开源大模型的面貌，也让普通设备运行万亿级知识量模型成为了现实。